Challenges Faced During Event Ticket System Development

Project: Event Ticket System – Full-Stack Event Management Platform
Date: May 12, 2026

During the development of the Event Ticket System, I encountered a range of technical and architectural challenges that required careful debugging, validation restructuring, and UI synchronization improvements. These challenges ultimately strengthened both the functionality of the application and my understanding of full-stack development practices with modern frameworks like Next.js and React.

1. Data Validation and Field Persistence

One of the most critical issues involved the validation layer silently stripping VIP seat capacity and VIP pricing fields before they reached the database. The VIP data was collected in the admin form and appeared to be successfully saved, but when retrieved from the database, the VIP fields were missing entirely. The form accepted the input, but the backend validation function was filtering out these fields before insertion.

To resolve this, I updated the validateEventInput() function in lib/validation.ts to explicitly parse and preserve vipSeatCapacity and vipPrice fields using non-negative number conversion. This experience emphasized that validation logic acts as a critical infrastructure layer even with a complete backend model, data will not persist if validation filters out fields before they reach persistence. Testing data end-to-end from form input through database to display became essential.

2. Cascade Operations and Data Integrity

Event deletion with existing bookings created a blocking conflict that prevented admins from removing events. The system returned to a 409 Conflict error, but no mechanism existed to handle the cascade of related bookings and refunds. Users needed a way to delete events, but the system rightfully blocked deletion to protect booking data.

I implemented a force delete flag that enables cascade deletion: when an admin force-deletes an event with bookings, the system automatically creates refunds for all affected tickets, updates booking statuses, and generates notifications to inform users of the event cancellation. This required implementing a Notification model, creating a refund workflow with status tracking, and ensuring data consistency across multiple operations. This highlighted the importance of designing cascade operations with atomicity in mind from the architectural phase.

3. State Management and UI Synchronization

VIP information existed in the database and displayed correctly on the admin panel but was completely absent from customer-facing pages (browse events and event detail pages). Despite the admin panel showing VIP pricing and capacity bars correctly, the VIP option wasn't visible to users during event browsing or booking. This exposed a gap between backend completeness and frontend functionality.

I addressed this by adding VIP display components to the browse page with FaStar icons and pricing badges and updated the event detail page to show both Standard and VIP ticket options with separate pricing and availability. Using the already-working admin panel as a template prevented duplicate bugs. This demonstrated that backend completeness does not guarantee frontend functionality features must be tested end-to-end across all user-facing pages.

4. Image URL Persistence from Internet Links

Event images provided via internet URLs were being validated but not saved to the database. Admins could enter image links during event creation, but the imageUrl field remained null in the database, causing images to never display on event cards or detail pages.

I resolved this by updating the validation function to trim and preserve the imageUrl field, adding it to the returned validation object. The fix involved recognizing that the validation layer was complete but incomplete it was processing the field structurally but not including it in the output. This reinforced the principle that validation must explicitly pass through all expected fields, not just check their validity.

5. QR Code Visibility and Display Issues

QR codes were being generated for bookings but not displayed on the user dashboard, preventing check-in functionality. The QR code generation logic existed, but the display component wasn't rendering the data URL properly.

I fixed this by ensuring QR code data URLs were included in booking responses from the API and properly displayed on the dashboard component. This required verifying the complete data flow from booking creation through database storage to frontend rendering.

6. Event Deletion Race Conditions and Loading States

When admins deleted events, the UI sometimes displayed stale data or didn't refresh properly. Clicking deleted while the request was in flight could cause multiple submissions or inconsistent state.

I implemented loading states on the delete button, prevented multiple submissions with a flag, and ensured the event list refreshed immediately after successful deletion. This prevented race conditions and provided clear visual feedback to users about operation status.

7. Admin Modal Form Layout Issues

The initial inline edit form for events pushed event cards down the page, causing layout shifting and poor user experience. The form took up significant vertical space and disrupted the visual flow of the admin panel.

I refactored the edit form into a fixed modal overlay with semi-transparent background, which floats above the content without affecting layout. This improved UX significantly while keeping code simpler than managing positional state changes. The modal pattern proved more scalable and provided better focus on the task at hand.

8. Responsive Design and Mobile Usability

Event cards and forms did not adapt well to mobile screens, making it difficult for users to browse and book tickets on smartphones and tablets. Touch targets were too small and layouts were cramped.

I implemented Tailwind CSS responsive utilities with mobile-first approach, adjusted grid layouts to single column on smaller screens, increased button sizes for touch targets, and optimized spacing for mobile devices. This ensured consistent usability across all screen sizes from 320px smartphones to desktop displays.

Conclusion

These challenges provided valuable learning experiences in validation architecture, cascade operations, state management, and responsive design. Key lessons included the importance of testing data persistence end-to-end across all layers, ensuring backend operations account for related data, synchronizing UI components across multiple pages for consistency, recognizing that validation acts as infrastructure not just error checking, and implementing feedback mechanisms like loading states to prevent race conditions. Addressing these issues not only improved the functionality of the Event Ticket System but also resulted in a more robust, maintainable, and user-friendly application that successfully handles complex operations like VIP seating, refunds, and event management.